

Freelancer Finance OS

Technical Design — HLD & LLD

Architecture, database schema, API design, sequence flows, and technology decisions

Document 2 of 2 — Technical Design
(see Document 1 for Product Roadmap & Execution Plan)

Backend: Django + DRF + PostgreSQL | Frontend: React + TypeScript

1. Technology Stack Decision

1.1 Backend — Django + Django REST Framework + PostgreSQL

This choice is already fixed by your preference, and it fits the project well: Django's batteries-included approach (admin panel, ORM, auth) minimizes boilerplate for a solo developer, Django REST Framework (DRF) gives a mature, well-documented way to build the JSON API, and PostgreSQL is a robust, free, well-supported relational database — a good fit for inherently relational data like invoices, clients, and financial records.

1.2 Frontend — React + TypeScript vs Angular

Recommendation: React + TypeScript. Here is the honest comparison for your specific situation (1 year of experience, solo, part-time, needs to move fast):

Factor	React + TypeScript	Angular
Learning curve	Gentler — just components + hooks + a router/data-fetching library	Steeper — its own DI system, RxJS observables, decorators, opinionated structure
Time to first working screen	Faster for a solo dev, especially with 1 year of experience	Slower initial ramp-up due to more required concepts upfront
Ecosystem for this project	Huge component library choice (MUI, shadcn/ui) speeds up building dashboards/forms	Smaller but capable ecosystem (Angular Material); more rigid conventions
Fits distribution strategy	Pairs naturally with Next.js if SEO landing pages are needed later for content marketing	Not SEO/content-oriented by default; more suited to large internal apps
Hiring later (Phase 5)	Much larger talent pool in India to hire from when scaling	Smaller pool, often more senior/expensive
Best fit when...	Small-to-mid team, fast iteration, product-led growth, content-driven distribution	Large enterprise teams, strict conventions needed across many developers

Angular is a fine framework, but it's optimized for large teams that benefit from its opinionated structure — that's not your situation right now. React + TypeScript gets you shipping faster with your current experience level, and keeps the door open to Next.js later if SEO-driven landing pages become important for distribution (see Document 1, Section 8).

1.3 Supporting Tools & Services

Purpose	Choice	Why
API auth	djangoRESTframework-simplejwt (JWT)	Standard, well-documented, stateless auth suited to a SPA frontend
PDF generation	WeasyPrint (Python, HTML/CSS → PDF)	Runs inside Django, lets you design invoice PDFs with familiar HTML/CSS
Payments	Razorpay (Payment Links + Webhooks API)	India-first, UPI support, no upfront integration cost
Transactional email	Resend or Brevo (free tier)	Reliable delivery for invoice emails/reminders at MVP volume

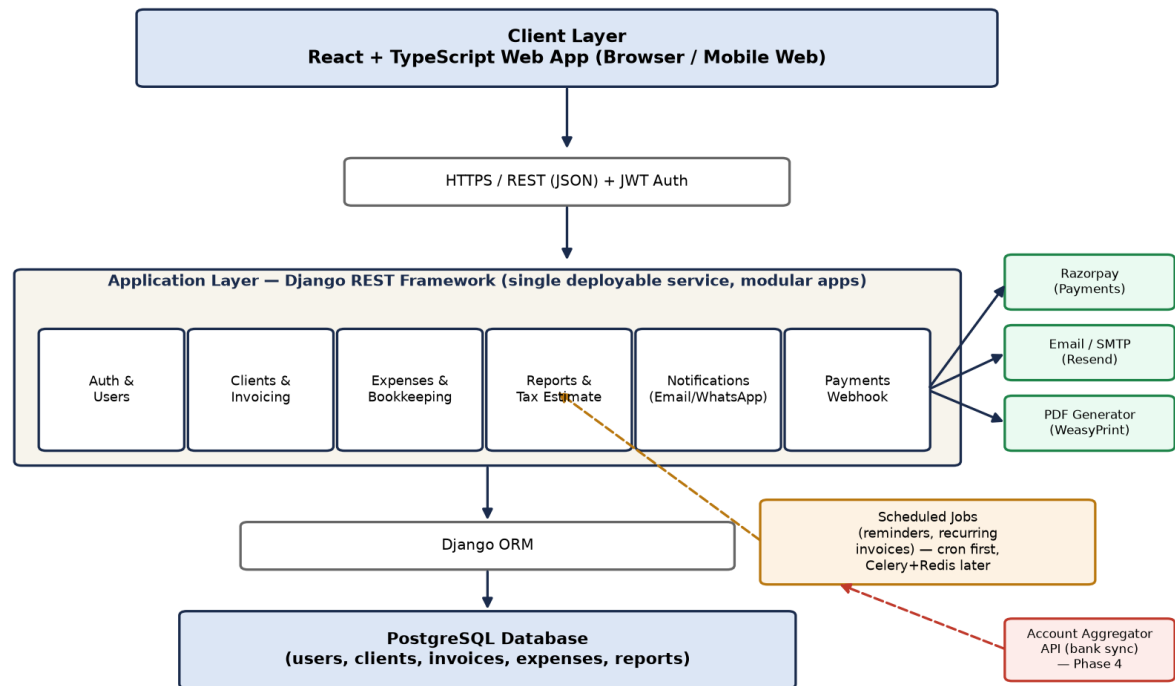
Purpose	Choice	Why
Background/scheduled jobs (MVP)	Django management commands + host-level cron	Avoids the operational cost of Celery+Redis until real scale needs it
Background jobs (Phase 4+)	Celery + Redis	Needed once volume/retries justify a proper task queue
State management (frontend)	React Query (server state) + React Context (light UI state)	Simpler than Redux for this app's scale; React Query handles caching/loading/error states well
Styling/UI components	Tailwind CSS + a component set (shadcn/ui)	Fast to build consistent, clean UI solo without a designer
Hosting (MVP)	Render or Railway (API+DB), Vercel/Netlify (frontend)	Free/low-cost tiers fit the ₹10,000 budget

2. High-Level Design (HLD)

2.1 Architecture Diagram

A single Django REST Framework service (modular internal apps, not separate microservices — unnecessary complexity for this stage) backed by PostgreSQL, called by a React + TypeScript single-page application. External services are called from the backend only, never directly from the frontend, so API keys/secrets never reach the browser.

High-Level Architecture — Freelancer Finance OS



Hosting (MVP budget): Render/Railway free-hobby tier for API + Postgres, Vercel/Netlify free tier for frontend, Cloudflare DNS. Total infra cost target: ~₹0-500/month until there is paying revenue.

2.2 Component Responsibilities

Component	Responsibility
Auth & Users	Signup/login, JWT issuance/refresh, password reset, user roles
Clients & Invoicing	Client CRUD, invoice creation, tax-type calculation, invoice numbering, PDF trigger
Expenses & Bookkeeping	Expense CRUD, categories, CSV import parsing/mapping
Reports & Tax Estimate	P&L calculation, GST/income-tax estimate logic, data export
Notifications	Templated emails (invoice sent, payment received, reminders)
Payments Webhook	Razorpay payment-link creation, webhook signature verification, Payment record updates
PDF Generator (WeasyPrint)	Renders an HTML invoice template into a PDF file, stored and linked to the Invoice record
Scheduled Jobs	Daily/weekly jobs: overdue-invoice reminders, recurring invoice generation (Phase 4)

2.3 Deployment View (MVP Budget)

- **Frontend:** React + TypeScript build deployed as static files on Vercel or Netlify (free tier) — global CDN included.
- **Backend:** Django + DRF deployed on Render or Railway's free/hobby tier as a single web service.
- **Database:** Managed PostgreSQL on the same platform (Render/Railway free tier), or Neon/Supabase free tier as an alternative.
- **File storage:** generated invoice PDFs stored on the same host's disk initially, migrating to S3-compatible object storage (e.g. Cloudflare R2, which has a generous free tier) once volume grows.
- **DNS/CDN:** Cloudflare (free) in front of the domain for DNS management and basic protection.
- **Scheduled jobs:** a Django management command triggered by the hosting platform's built-in cron/scheduled-job feature (most of the platforms above offer this on free tiers) — no separate worker service needed yet.

This deployment can realistically run at ₹0-500/month until there is paying revenue, matching the ₹10,000 starting budget in Document 1.

2.4 Non-Functional Requirements

Concern	Approach at MVP stage
Security	HTTPS everywhere (free via hosting platform/Cloudflare), JWT with short-lived access tokens + refresh tokens, Django's built-in password hashing (PBKDF2/Argon2)
Data privacy	GSTIN/PAN and financial data are sensitive — restrict admin access, avoid logging sensitive fields, encrypt backups
Availability	Single-region deployment is acceptable at MVP scale; no need for multi-region redundancy yet
Backups	Daily automated PostgreSQL backups (most managed providers include this even on free/low tiers) — verify restore works at least once
Scalability	Not a concern at MVP scale (tens to low hundreds of users); Django+Postgres comfortably handles this on a single small instance
Auditability	Key financial actions (invoice created, payment recorded, invoice edited) are logged with timestamps for dispute resolution

3. Low-Level Design (LLD)

3.1 Django Project / App Structure

One Django project with focused, modular apps — each maps directly to a component from Section 2.2. This keeps the codebase organized without the operational overhead of separate microservices.

```
freelancer_finance_os/  
  config/                # Django project settings, urls.py, wsgi/asgi  
  accounts/              # User model, auth, JWT views, BusinessProfile  
  clients/               # Client model + CRUD API  
  invoicing/             # Invoice, InvoiceItem, Payment models + tax logic  
  expenses/              # ExpenseCategory, Expense, BankStatementImport  
  reports/               # P&L calculation, TaxEstimate model, export views  
  notifications/         # Email templates, NotificationLog, sending logic  
  payments/              # Razorpay integration, webhook handler  
  common/                # Shared utilities (currency conversion, PDF rendering helpers)  
  manage.py
```

3.2 Database Schema

Full field-level schema for each core table (PostgreSQL types shown):

User (extends Django's AbstractUser)

Field	Type	Notes
id	BigAutoField (PK)	Primary key
email	CITEXT / EmailField, unique	Used as the login identifier
password	CharField (hashed)	Handled by Django's auth system
full_name	CharField(150)	
role	CharField, choices	'freelancer' (default) or, later, 'accountant' for Phase 5
created_at	DateTimeField (auto_now_add)	

BusinessProfile

Field	Type	Notes
id	BigAutoField (PK)	
user	OneToOneField(User)	One business profile per user
business_name	CharField(255)	
gstn	CharField(15), nullable	Null if not yet GST-registered
pan	CharField(10)	
state	CharField(100)	Used to determine CGST/SGST vs IGST
is_gst_registered	BooleanField	
invoice_prefix	CharField(10), default 'INV'	Used to build sequential invoice numbers
next_invoice_seq	IntegerField, default 1	Incremented atomically per invoice created

Client

Field	Type	Notes
id	BigAutoField (PK)	
user	ForeignKey(User)	
name	CharField(255)	
email	EmailField	
billing_address	TextField	
country	CharField(100)	'India' or foreign country name
state	CharField(100), nullable	Required if country is India
client_gstin	CharField(15), nullable	
is_international	BooleanField, default False	Drives export-supply invoice logic

Invoice

Field	Type	Notes
id	BigAutoField (PK)	
user	ForeignKey(User)	
client	ForeignKey(Client)	
invoice_number	CharField(30), unique per user	e.g. INV-2026-0001
issue_date	DateField	
due_date	DateField	
currency	CharField(3), default 'INR'	ISO currency code
exchange_rate_to_inr	DecimalField(10,4) , default 1	Locked at issue time for foreign-currency invoices
tax_type	CharField, choices	'CGST_SGST' / 'IGST' / 'EXPORT_ZERO_RATED'
subtotal	DecimalField(12,2)	
cgst_amount / sgst_amount / igst_amount	DecimalField(12,2)	Only the relevant fields are non-zero
total_amount	DecimalField(12,2)	
status	CharField, choices	'draft' / 'sent' / 'paid' / 'overdue'
pdf_file	FileField, nullable	Path to the generated PDF
created_at / updated_at	DateTimeField	

InvoiceItem

Field	Type	Notes
id	BigAutoField (PK)	
invoice	ForeignKey(Invoice)	
description	CharField(255)	
hsn_sac_code	CharField(10)	
quantity	DecimalField(10,2)	
unit_price	DecimalField(12,2)	
tax_rate_percent	DecimalField(5,2)	e.g. 18.00 for 18% GST
amount	DecimalField(12,2)	quantity × unit_price, pre-tax

Payment

Field	Type	Notes
id	BigAutoField (PK)	
invoice	ForeignKey(Invoice)	
amount	DecimalField(12,2)	

Field	Type	Notes
payment_date	DateTimeField	
method	CharField	'razorpay' / 'manual'
razorpay_payment_id	CharField(100), nullable	
status	CharField, choices	'pending' / 'captured' / 'failed'

ExpenseCategory

Field	Type	Notes
id	BigAutoField (PK)	
user	ForeignKey(User)	
name	CharField(100)	e.g. 'Software', 'Travel'
is_default	BooleanField	True for the seeded default categories

Expense

Field	Type	Notes
id	BigAutoField (PK)	
user	ForeignKey(User)	
category	ForeignKey(ExpenseCategory)	
vendor_name	CharField(255)	
amount	DecimalField(12,2)	
expense_date	DateField	
source	CharField, choices	'manual' / 'csv_import'
bank_statement_import	ForeignKey(BankStatementImport), nullable	Set only for CSV-imported rows
receipt_file	FileField, nullable	
notes	TextField, blank	

BankStatementImport

Field	Type	Notes
id	BigAutoField (PK)	
user	ForeignKey(User)	
file_name	CharField(255)	
status	CharField, choices	'processing' / 'completed' / 'failed'
imported_at	DateTimeField (auto_now_add)	

TaxEstimate

Field	Type	Notes
id	BigAutoField (PK)	

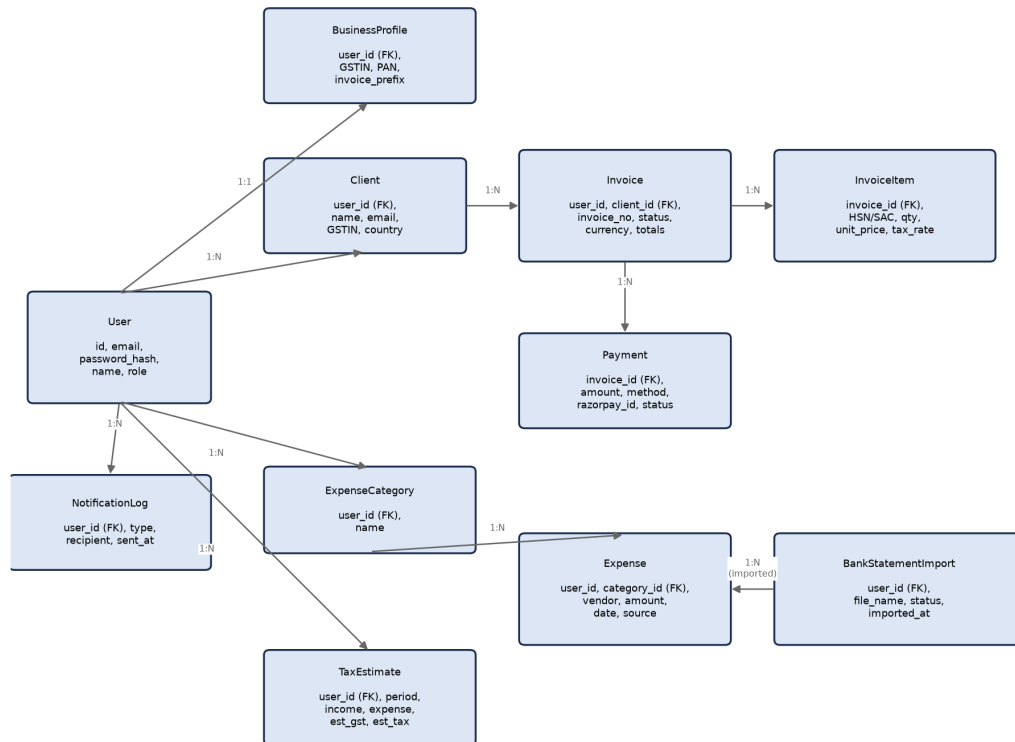
Field	Type	Notes
user	ForeignKey(User)	
period_start / period_end	DateField	
total_income	DecimalField(14,2)	
total_expense	DecimalField(14,2)	
estimated_gst_liability	DecimalField(12,2)	
estimated_income_tax	DecimalField(12,2)	Based on Section 44ADA presumptive rate

NotificationLog

Field	Type	Notes
id	BigAutoField (PK)	
user	ForeignKey(User)	
type	CharField	'invoice_sent' / 'payment_received' / 'reminder'
recipient	EmailField	
sent_at	DateTimeField	
status	CharField	'sent' / 'failed'

3.3 Entity Relationship Diagram

Simplified Entity Relationship Diagram — Core Data Model



3.4 API Endpoint Specification

Method	Path	Description	Auth
POST	/api/auth/register	Create a new user + BusinessProfile	None
POST	/api/auth/login	Obtain JWT access + refresh tokens	None
POST	/api/auth/refresh	Refresh an access token	Refresh token
GET/PATCH	/api/business-profile	View/update the current user's business profile	JWT
GET/POST	/api/clients	List or create clients	JWT
GET/PATCH/DELETE	/api/clients/{id}	Retrieve/update/delete a client	JWT
GET/POST	/api/invoices	List or create invoices	JWT
GET/PATCH	/api/invoices/{id}	Retrieve/update an invoice	JWT
POST	/api/invoices/{id}/send	Email the invoice PDF + payment link to the client	JWT
GET	/api/invoices/{id}/pdf	Download the generated invoice PDF	JWT
GET/POST	/api/expenses	List or create expenses	JWT
POST	/api/expenses/import-csv	Upload a bank statement CSV for parsing	JWT
GET/POST	/api/expense-categories	List or create expense categories	JWT
GET	/api/reports/profit-loss?period=	Get P&L for a given period	JWT
GET	/api/reports/tax-estimate?period=	Get GST/income-tax estimate for a period	JWT
POST	/api/webhooks/razorpay	Razorpay payment webhook receiver	Webhook signature

3.5 Example Request/Response — Create Invoice

Request: POST /api/invoices

```
{
  "client_id": 14,
  "issue_date": "2026-08-23",
  "due_date": "2026-09-06",
  "currency": "USD",
  "items": [
    {
      "description": "UI/UX design – August retainer",
      "hsn_sac_code": "998314",
      "quantity": 1,
      "unit_price": 500.00,
      "tax_rate_percent": 0
    }
  ]
}
```

Response: 201 Created

```
{
  "id": 87,
  "invoice_number": "INV-2026-0043",
  "client_id": 14,
  "currency": "USD",
  "exchange_rate_to_inr": 83.42,
  "tax_type": "EXPORT_ZERO_RATED",
  "subtotal": 500.00,
  "cgst_amount": 0, "sgst_amount": 0, "igst_amount": 0,
  "total_amount": 500.00,
  "status": "draft",
  "pdf_url": "/api/invoices/87/pdf",
  "payment_link": "https://rzp.io/i/abc123xyz"
}
```

3.6 Sequence Flow — Create Invoice, Send, and Get Paid

Flow Example — Create Invoice, Send, and Get Paid



3.7 Flow — Upload Bank Statement CSV and Categorize Expenses

- User uploads a CSV export from their bank/UPI app on the Expenses screen.
- Frontend sends the file to POST /api/expenses/import-csv; backend creates a BankStatementImport row with status 'processing'.
- Backend parses the CSV (column mapping: date, description, amount, debit/credit) and, for each debit row, creates a draft Expense linked to that BankStatementImport.
- A simple keyword-matching rule set suggests a category per row (e.g. 'AWS', 'GitHub' → Software; 'Uber', 'Ola' → Travel) — this can start as a hardcoded dictionary and improve over time.
- User reviews the imported rows in a confirmation screen, adjusts categories where the auto-suggestion is wrong, and confirms.
- Backend marks BankStatementImport as 'completed' and the confirmed Expense rows become part of the user's regular expense data feeding the P&L report.

3.8 Tax Calculation Logic (Core Business Rule)

This is the most important piece of domain logic in the entire product — get this right and validate it with a tax professional before launch.

```
def determine_tax_type(business_profile, client):
    if client.is_international:
        return "EXPORT_ZERO_RATED"          # zero-rated, requires LUT on file
    if client.state == business_profile.state:
        return "CGST_SGST"                  # intra-state supply
    return "IGST"                           # inter-state supply

def calculate_invoice_totals(items, tax_type):
    subtotal = sum(item.quantity * item.unit_price for item in items)
    if tax_type == "EXPORT_ZERO_RATED":
        return subtotal, 0, 0, 0, subtotal   # no GST charged on exports

    tax_total = sum(
        item.quantity * item.unit_price * (item.tax_rate_percent / 100)
        for item in items
    )
    if tax_type == "CGST_SGST":
        cgst = sgst = tax_total / 2
        igst = 0
    else: # IGST
        cgst = sgst = 0
        igst = tax_total

    total = subtotal + tax_total
    return subtotal, cgst, sgst, igst, total
```

3.9 Frontend Architecture

```
src/  
  api/                # Typed API client functions (one file per resource)  
  components/         # Reusable UI components (Button, Table, FormField, ...)  
  features/  
    auth/             # Login, register pages + auth state  
    clients/          # Client list, create/edit forms  
    invoices/         # Invoice list, create/edit, PDF preview  
    expenses/         # Expense list, manual entry, CSV import flow  
    reports/          # P&L dashboard, tax estimate view  
  hooks/              # Shared React Query hooks (useInvoices, useExpenses, ...)  
  routes/             # React Router route definitions  
  types/              # Shared TypeScript types/interfaces mirroring the API  
  App.tsx
```

Key screens for the Phase 1-3 MVP:

- Login / Register
- Business profile setup (onboarding)
- Client list + create/edit client
- Invoice list (filter by status) + create/edit invoice + PDF preview
- Expense list + manual entry form + CSV import wizard
- Dashboard: P&L summary + tax estimate + recent activity

3.10 Security Considerations

- **Passwords:** never stored in plain text — Django's default PBKDF2 hashing is sufficient at this stage.
- **JWT tokens:** short-lived access tokens (e.g. 15 min) + longer-lived refresh tokens (e.g. 7 days), refresh tokens stored in an HttpOnly cookie where possible to reduce XSS risk.
- **Webhook verification:** Razorpay webhook payloads must be verified against their signature header before trusting them — never update payment status from an unverified request.
- **Sensitive fields:** GSTIN/PAN are not secrets but are sensitive; avoid exposing them in logs or error messages.
- **File uploads:** validate CSV/receipt file types and size limits server-side before processing.
- **Rate limiting:** apply basic rate limiting on auth endpoints (DRF's throttling classes) to reduce brute-force risk.
- **Backups:** enable automated daily PostgreSQL backups from day one — financial data loss is not an acceptable risk even at MVP stage.

Appendix — Technical Glossary

HLD (High-Level Design): A description of the overall system architecture and how major components fit together, without implementation detail.

LLD (Low-Level Design): Detailed design of individual components: database schema, API contracts, class/module structure, and algorithms.

REST API: An architectural style for web APIs where resources (e.g. invoices, clients) are accessed via standard HTTP methods (GET, POST, PATCH, DELETE).

JWT (JSON Web Token): A compact, signed token used to authenticate API requests without the server needing to store session state.

ORM (Object-Relational Mapper): A library (Django's built-in ORM here) that lets you work with database rows as Python objects instead of writing raw SQL.

Webhook: An HTTP callback — a third-party service (like Razorpay) calls your API automatically when an event happens (e.g. a payment is captured).

SPA (Single-Page Application): A web app (like the React frontend here) that loads once and updates the page dynamically via JavaScript instead of full page reloads.

Migration (Django): A version-controlled file describing a change to the database schema, generated and applied via Django's migration system.

CI/CD: Continuous Integration/Continuous Deployment — automated testing and deployment pipelines (can be added later; not required for MVP).

Idempotency: A property where performing the same operation multiple times (e.g. a retried webhook) has the same effect as doing it once — important for payment webhook handling.